SMART HOME IoT TEMPERATURE PREDICTION USING MACHINE LEARNING

A beginner-friendly Edge/IoT mini project

Domain	Smart Home / IoT / Edge Computing
Language	Python
ML Algorithm	Decision Tree Classifier
Inputs	Temperature (°C), Humidity (%)
Output	Comfortable / Too Hot / Too Cold

Prepared as a simulated sensor-based demonstration. No physical IoT device is required.

1. Introduction

Smart homes use sensors to monitor conditions such as temperature and humidity. This project demonstrates a simple IoT/Edge workflow by generating sample sensor readings and using a machine-learning model to classify the room condition. The data is simulated in Python, so hardware is not required.

2. Objectives

• Generate a suitable synthetic IoT dataset.
• Train and save a machine-learning model.
• Simulate incoming sensor values.
• Predict room status quickly using the saved model.
• Display results through an interactive dashboard.

3. Tools and Technologies

Python, pandas, scikit-learn, joblib, Streamlit, Matplotlib, VS Code, and a CSV dataset.

4. Dataset Description

The dataset contains 1200 synthetic records and three columns: temperature_c, humidity_percent, and room_status. Temperature ranges from 10°C to 42°C, and humidity ranges from 20% to 95%. The target labels are created using simple illustrative rules. These labels are only for demonstrating the ML workflow and are not an official comfort or safety standard.

5. Methodology

1) Create simulated sensor readings. 2) Split data into 80% training and 20% testing. 3) Train a Decision Tree Classifier with maximum depth 5. 4) Evaluate it on held-out test data. 5) Save the model using Joblib. 6) Load the saved model in a Streamlit app. 7) Let the user enter sensor values or generate a random reading and display the prediction.

6. System Architecture

Simulated temperature/humidity → feature table → trained Decision Tree model → predicted room status → Streamlit dashboard. In a real deployment, simulated readings could be replaced by values from a sensor such as a DHT22 connected to an ESP32/Raspberry Pi. This project does not connect to physical hardware.

7. Model Evaluation

The Decision Tree model achieved 100.0% accuracy on the held-out test split of the synthetic dataset. Since the labels are generated from simple rules, this score mainly confirms that the model learned those rules; it does not prove real-world performance.

Classification report:

```
precision recall f1-score support

Comfortable 1.00 1.00 1.00 77
Too Cold 1.00 1.00 1.00 62
Too Hot 1.00 1.00 1.00 101

accuracy 1.00 240
macro avg 1.00 1.00 1.00 240
weighted avg 1.00 1.00 1.00 240
```

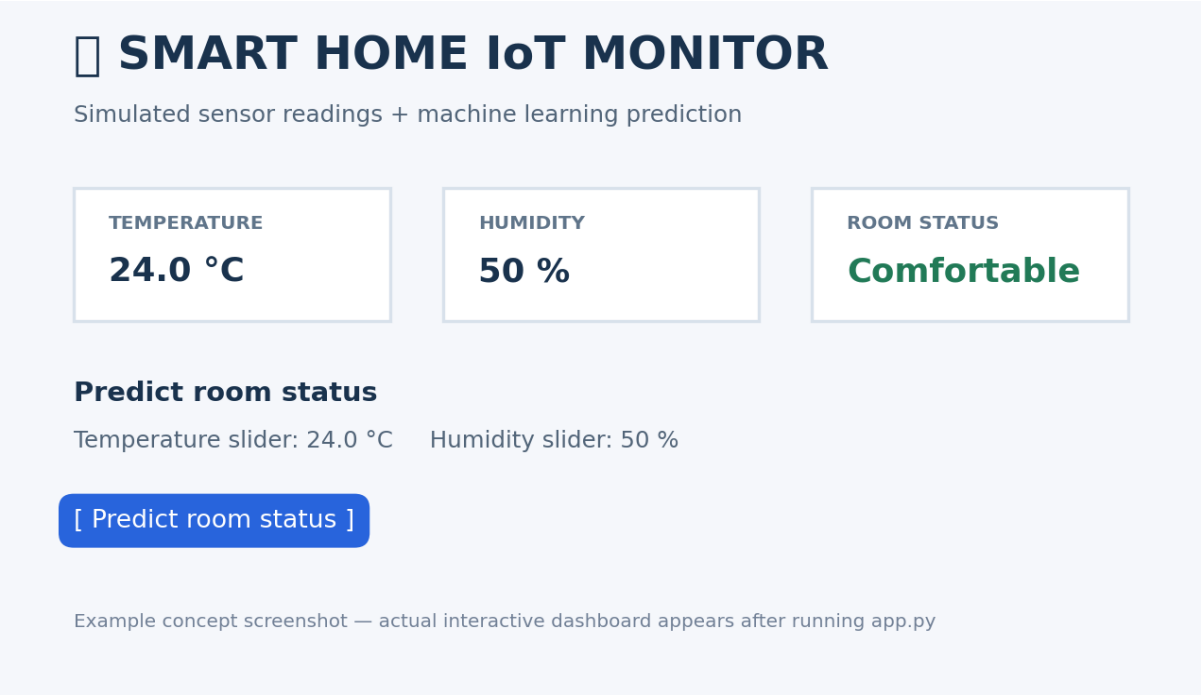
8. How to Run

Open the project folder in VS Code. In the terminal, run:
`python -m venv .venv`
Activate the environment on Windows with `.venv\Scripts\Activate.ps1`, then run:
`pip install -r requirements.txt`
`python train_model.py`
`streamlit run app.py`
Use the dashboard sliders and click “Predict room status.”

9. Results and Screenshots

The application accepts temperature and humidity values, loads the registered/saved model file, and displays one of three predicted classes. The following image is an illustrative mock-up of the dashboard, not a screenshot captured from a live Streamlit session.

Dashboard example



10. Limitations and Future Scope

Limitations: data is synthetic; no real sensor is connected; labels use simple rules; and the dashboard is a local demonstration. Future improvements could use real sensor data, add timestamps and charts, connect to MQTT, and deploy the app on a Raspberry Pi or edge gateway.

11. Conclusion

This project demonstrates a complete beginner-level IoT/Edge machine-learning pipeline: dataset preparation, model training, model saving, simulated live input, prediction, and dashboard display. It is easy to run and can be extended to real sensors later.

12. Five-Minute Demonstration Script

0:00–0:45 — Introduce the problem: monitor room temperature and humidity.
0:45–1:30 — Show the CSV dataset and explain the two input features and target label.
1:30–2:15 — Run `python train_model.py` and show model evaluation and saved model path.
2:15–3:45 — Run `streamlit run app.py`, change the sliders, and click Predict.

3:45–4:30 — Generate a simulated sensor reading and show the prediction.
4:30–5:00 — Explain limitations and future scope.

Appendix: Project Files

app.py — dashboard; train_model.py — model training; iot_sensor_dataset.csv — dataset; model/smart_home_model.joblib — trained model; requirements.txt — dependencies; screenshots/ — illustrative screenshots.